

CITS5501 Project Phase 1 Report

Repository: [CITS5501 Group 1 Repository](#)

Group members:

- Juan Yerima Yovian - Student ID: 24911605 - Github: jyovian
- Jinghan Wu - Student ID: 24289151 - Github: Jinghan00
- Shee Wang - Student ID: 24368932 - Github: sheewang
- Gelani Nimit Sureshbhai - Student ID: 24765784 - Github: Fighterdx
- Laura Acosta Vargas - Student ID: 23802255 - Github: laura-camila-av

Planning and organization

1. Where is your group's Collaboration & Workflow Plan maintained? Is it in the Git repository, or elsewhere?

Our Collaboration & Workflow Plan is maintained in the Git repository at [docs/collaboration-workflow-plan.md](#). Keeping it with the project makes the plan accessible to every member, records its revision history, and allows proposed changes to be reviewed through the same pull-request process used for the rest of the project.

2. What tool or approach are you using to track assigned tasks and completion? Why did you choose it? Has anyone in the group used it before?

We are using GitHub Projects that links directly to our repository's Issues. This was chosen because it's a simple and convenient way of tracking our progress. No extra app or account or tool required. Everyone already has a GitHub account. It also links directly to commits/PRs. Some members of the group have used GitHub Projects before, while others are already familiar with basic GitHub functions.

Language and tooling

3. Which language has your group chosen? Have any members of the group not used it — and if so, how will they get familiar with it? What factors influenced your choice of implementation language? Rank the top three in order of importance.

Our chosen language for this project is **Python**. We chose it mainly because web scraping can involve unexpected HTML and different document formats. Python allows us to develop and debug this type of software relatively quickly.

Top 3 factors:

1. **Library support.** Python has mature and well-documented libraries for many stages of this project, from fetching, HTML parsing, document extraction, etc. These help reduce the amount of custom code we will need to write and test.
2. **Team members' experience.** Every member of this group has experience with Python.

3. **Ease of debugging.** If the scraper fails because of unexpected or changing website behaviour, Python should allow us to identify and modify the affected code relatively quickly.

What advantages do you expect your chosen language to give your team? What disadvantages or risks do you expect?

Advantages:

- Allows us to develop features relatively quickly.
- Strong library support for web scraping and document processing.

Disadvantages:

- Python's dynamic typing means that some type-related errors may only be detected at runtime.
- Using several third-party libraries may introduce dependency management issues.

If you had to implement this project in the other language, what do you think would be most difficult? What might be easier?

If we had to use another language, such as Java, it would probably be harder to deal with all the different ways things can go wrong when scraping (bad HTML, missing files, network errors, etc). Java would require us to handle these exceptions really explicitly, which can be good for safety but would slow us down a lot. Plus, Java introduces boilerplate code and complexity for those who aren't familiar with it. However, Java's static type system and compiler checks could make some errors easier to detect before the program is run.

4. What build/dependency management tool will you use (e.g. uv or poetry for Python; Gradle or Maven for Java; something else)? Has anyone in the group used it before — or will this be the first time setting up a project with it?

We will use **uv** to manage our dependencies and virtual environment. We chose it because it's fast, keeps a lockfile so everyone in the group installs the exact same package versions, and it's already available in the environment our project will be tested in. This will be the first time most of the group has used uv specifically, although everyone is familiar with tools like pip or venv, so picking it up shouldn't take long.

Technical risks

5. What parts of a web scraper do you expect could most easily fail, and how?

The parts most likely to fail are the network requests, HTML parsing, and document downloading. Network requests may fail because of timeouts, server errors, or temporary connection problems. HTML parsing may fail if the website structure changes or if the page contains malformed HTML. Document downloads may also fail because of broken links, missing files, unexpected file types, or corrupted documents.

6. Which external factors could make your software stop working after it has been completed?

The software could stop working if the website changes its structure, URLs, or document formats. It could also be affected by new login requirements, stricter rate limits, or changes in the server. Updates to third-party libraries may also cause compatibility problems.

7. Which components of your project do you think will be easiest to test? Which do you expect will be hardest?

- The easiest components to test will probably be functions that process fixed inputs, such as URL handling, file type detection, text processing, and query logic. These can be tested using known inputs and expected outputs.
- The hardest components to test will be network requests and interactions with the real website. These depend on external factors such as connection problems, server errors, rate limits, and changes to the website. These behaviours are harder to reproduce consistently in tests.

8. What assumptions is your software likely to make about the website it scrapes? How could those assumptions be violated?

Our software will probably assume that the website structure stays mostly consistent, that document links work, and that file types can be identified correctly. These assumptions could fail if the website layout changes, links are broken, files have unexpected formats, or the server returns unusual responses.

Team practices

9. Suppose a serious bug is discovered two days before the deadline. How would your team investigate and respond?

If a serious bug is found two days before the deadline, we would first reproduce it and identify the affected component. The issue would be given high priority and assigned to appropriate team members. If necessary, non-essential work would be paused. After fixing the bug, we would run the relevant tests and regression tests, review the change, and merge it only after confirming that the problem is resolved.

10. What will convince your team that a feature is "finished"?

A feature will be considered finished when it meets the project requirements, works as expected, has appropriate tests, and passes all relevant tests. It should also be reviewed by another team member before being merged into the main branch.

11. If two group members disagree about whether software is "working correctly", how will you resolve that disagreement?

If two group members disagree about whether the software is working correctly, we will first check the project specification and any official clarifications. We will then create a test case for the disputed behaviour and compare the actual result with the expected result. If the specification is still unclear, we will discuss the issue with the teaching staff and document any necessary assumptions.

Code quality

12. Python groups: Will you use a type checker (e.g. mypy or pyright), and if so, will it run in CI or just periodically/manually? Has anyone used one before? Do you plan to use a linter (e.g. pylint, black)?

We will use **mypy** as our type checker, and it will run automatically as part of our CI pipeline on every push and pull request, rather than just being run manually. This means type errors get caught early before the code is even reviewed, rather than relying on individual members remembering to run it themselves.

For linting and formatting, we will use **ruff**, which covers what tools like pylint and black would normally do separately, but in a single tool. Some of the group has not used mypy or ruff before, so we expect a learning curve.

13. Suppose a group member's contribution doesn't pass your linting or type checking checks. What will be the consequences? Will merges be blocked if these checks fail, or will the checks be advisory only?

If a contribution does not pass the agreed linting or type checking checks, the author should fix the reported issues before merging. In most cases, failed checks should block the merge, although minor advisory warnings may be reviewed by the team if they do not affect correctness or maintainability.

Testing framework

14. What test framework will you use (e.g. pytest for Python; JUnit 5 for Java; something else)? Has the group used it before, or only heard of it?

We will use **pytest**. We chose it over Python's built-in **unittest** module because it lets us write tests with less boilerplate, and some features make it easy to run the same test against many different input values, which suits the kind of equivalence-class and boundary testing we'll need for a scraper handling lots of different edge cases. Some of the group have not used pytest before.

CI / automation

15. Will you set up continuous integration (e.g. GitHub Actions) to run your test suite automatically, or run tests manually before submission? Have any members of your group used continuous integration before?

We will use GitHub Actions to automatically run our linter, type checker, and test suite every time someone pushes code or opens a pull request. We chose to automate this rather than run tests manually, since manual testing is easy to forget or skip, and automation means every change gets checked the same way without relying on someone remembering to do it. Most members of the group have not set up CI before, so this will be a new process for the team to learn.

Testing mindset

16. Suppose your scraper is finished and submitted. Now imagine another student is trying to break your software (get it to exhibit failures or unexpected behaviour). What might they try first?

They might first try invalid or unusual inputs, such as broken URLs, empty queries, unsupported file types, malformed documents, or pages with unexpected HTML. They may also try situations such as timeouts, server errors, duplicate links, or very large documents to see whether the scraper handles them correctly without crashing.

17. How will you know that your software is producing the correct output, rather than merely some output?

We will use test cases with known expected results and compare them with the actual output of the program. We can also manually check some retrieved documents and query results against the source website to

confirm that the software is returning the correct information.

18. What evidence would increase your confidence that your software is reliable?

We would have more confidence in the software if it consistently passes a wide range of tests, including normal cases, edge cases, and expected failure cases. Successful CI runs and code reviews by other team members would also increase our confidence in its reliability.

19. At this stage of the project, what do you think will be the biggest challenge in producing reliable software?

The biggest challenge will likely be dealing with factors outside our control, especially changes or failures in the external website. Broken links, network problems, unexpected responses, and changes in page structure may make it difficult to ensure the scraper behaves reliably in every situation.

Process and standards

20. What will be your code review process — does someone other than the author review changes before merging, or is this informal?

Changes will be made on separate branches and submitted through Pull Requests. Other team members will review the changes before they are merged into the protected main branch. Reviewers will check the code, tests, and whether the change meets the project requirements. Any issues found during review should be fixed before approval and merging.

Assumptions

- **A001:** We're assuming that the policy library website we'll eventually be provided is a fairly typical server-rendered website, meaning that the papers are linked straight from regular HTML pages rather than being loaded via JavaScript. This is a guess because we won't see the site until after Phase 1, but it's the more typical scenario and the foundation of our intended tools, which combines HTML parsing with requests style fetching. If it turns out that the website uses a lot of JavaScript, we would have to replace it with something like a headless browser.
- **A002:** We're assuming that the website's materials are open to the public and don't require a login. If identification turns out to be necessary, we haven't yet scoped that additional work because it wasn't taken into account in our risk planning for Q5–Q8.
- **A003:** We're assuming that scraping this specific website is permitted due to its robots.txt, conditions of use, or the fact that it was put up just for this unit. This concerns for how carefully our scraper needs to act (request rate, retry logic), which relates into our response for Q5.
- **A004:** We're assuming the GitHub Actions runners we use for CI will have normal outbound internet access, in case any of our tests end up hitting the real site rather than a mocked version of it. If that's not reliable, we'll need to rely more on mocked responses for CI and treat live-site tests as something we only run locally.
- **A005:** The project brief only guarantees that `python3`, `uv`, and `openjdk-21-jdk` are installed in the marking environment (see section 2.5). We're assuming that's enough that `mypy`, `ruff`, and `pytest` don't need to be separately installed, because `uv` will pull them in from our project's lockfile automatically.

- **A006:** We're assuming the scope stays limited to the three document formats named in the brief PDF, Word, and RTF. If the real site also serves other formats (e.g. plain text or OpenDocument files), we'd treat those as out of scope unless told otherwise, rather than trying to support everything we might encounter.